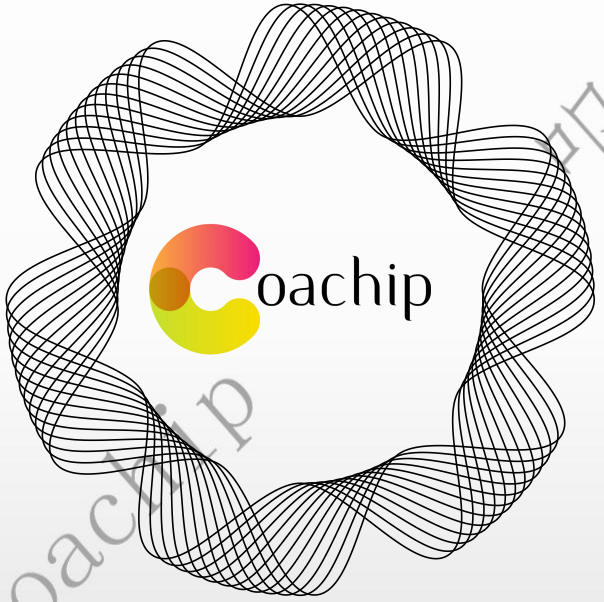


SV Assertions 断言

叩特电子Coachip

课程内容及目标

课程内容：

- 了解断言的作用和种类
- 了解断言的基本语法

目标：

- 能理解断言
- 能写一些简单断言

叩特电子Coachip

SV assertion简介

断言（Assertion）可以被放在RTL设计中，方便在仿真时查看异常情况。当异常出现时，断言会报警。一般在数字电路设计中都要加入断言，断言占整个设计的比例应不少于30%。

断言的作用：

- 检查特定条件或事件序列的出现情况。
- 提供功能覆盖

断言的种类：

- 立即断言 Immediate Assertions
- 并发断言 Concurrent Assertions

叩特电子Coachip

SV assertion在code中的位置

```
module inline(
  input        clk ,
  input        a   ,
  input        b   ,
  input  [7:0] d1  ,
  input  [7:0] d2  ,
  output [7:0] d
);
  reg [7:0] d = 7'h0;

  always @(posedge clk)
  begin
    if(a)1
      d <= d1;
    if(b)
      d <= d2;
  end
```

RTL代码

```
  property p_mutex;
    @(posedge clk) not (a && b);
  endproperty

  a_mutex: assert property (p_mutex);

endmodule
```

SVA

RTL代码中

```
`include "./top/define.sv"
`include "../std_lib/common_interface.sv"

module top;

  //...

  // clk
  wire reg_clk;
  // rst
  wire reg_rst;

  //...

assertion_2 x_assertion(
  .clk    (reg_clk),
  .rst_n  (reg_rst)
);
`ifdef SIM_ASSERTION_ON

//a33
  a33a1: assert property( @ (posedge reg_clk) $onehot(x_assertion.bus) );

  a33b1: assert property( @ (posedge reg_clk) $onehot0(x_assertion.bus) );

  a33c1: assert property( @ (posedge reg_clk) $isunknown(x_assertion.bus) );

  a33d1: assert property( @ (posedge reg_clk) $countones(x_assertion.bus) );

`endif
```

仿真环境中

oachip

立即断言 Immediate Assertions

立即断言检查当前仿真时间的条件，相当于 if else ， 需要放在过程块中。

语法：

```
labels: assert(expression) action_block;
```

其中：

- action block 操作块在断言表达式的求值之后立即执行
- 操作块指定在断言成功或失败时采取什么操作
- action_block: pass_statement; else fail_statement;

例子：

```
assert(expression) $display("expression evaluates to true");
else $fatal("expression evaluates to false");
```

叩特电子Coachip

立即断言 Immediate Assertions

由于断言表达式中所断言的条件必须为真，因此断言的失败将具有与之相关的"严重"程度。

默认情况下，断言失败的严重程度是一个error。

还有其他可选的fail statement:（严重等级依次递减）

- $fatal ： a run-time fatal.
- $error ： a run-time error.
- $warning ： a run-time warning.
- $info ： information.

叩特电子Coachip

立即断言 Immediate Assertions

```
module asertion_ex;
   bit clk,a,b;
   always #5 clk = ~clk;   //clock generation
   initial begin
          a=1;
          b=1;
     #15 b=0;
     #10 b=1;
          a=0;
     #20 a=1;
     #20;
     $finish;
   end
   always @(posedge clk) check_a_and_b:assert(a&&b); //Immediate assertion
endmodule
```

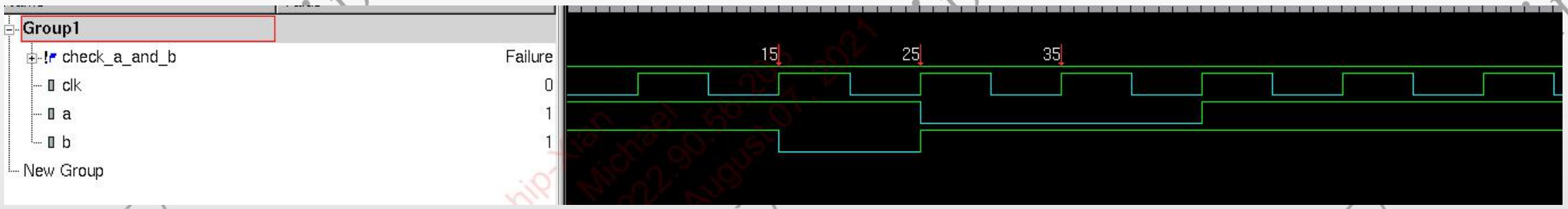
叩特电子Coachip

立即断言 Immediate Assertions

asertion_ex.check_a_and_b: started at 15ps failed at 15ps Offending ‘ (a&&b)’

asertion_ex.check_a_and_b: started at 25ps failed at 25ps Offending ‘ (a&&b)’

asertion_ex.check_a_and_b: started at 35ps failed at 35ps Offending ‘ (a&&b)’



即特电子Coachip

立即断言 Immediate Assertions

```
module asertion_ex;
  bit clk,a,b;
  always #5 clk = ~clk;   //clock generation
  initial begin
        a=1;
        b=1;
    #15 b=0;
    #10 b=1;
        a=0;
    #20 a=1;
    #20;
  $finish;
  end
  always @(posedge clk) begin
     check_a_and_b:assert(a&&b) $display("a&&b is true");
                                else $fatal("a&&b is false");
  end
endmodule
```

叩特电子Coachip

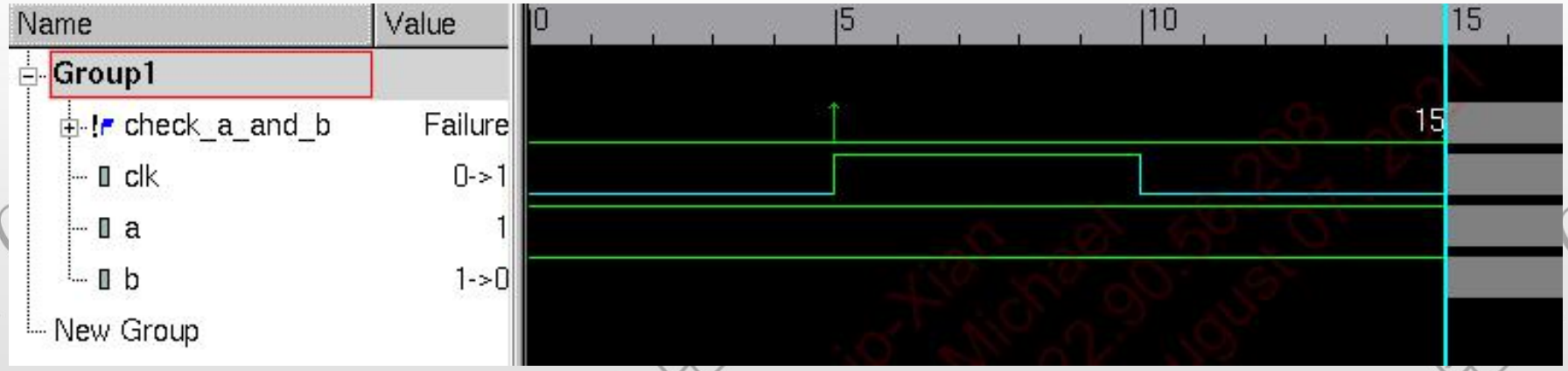
立即断言 Immediate Assertions

a&&b is ture

asertion_ex.check_a_and_b: started at 15ps failed at 15ps Offending ‘ (a&&b)’

Fatal: assertion_ex.check_a_and_b: at time 15 ps

a&&b is false



oachip

立即断言 Immediate Assertions

```
module asertion_ex;
  bit clk,a,b;
  always #5 clk = ~clk;   //clock generation
  initial begin
        a=1;
        b=1;
   #15 b=0;
   #10 b=1;
        a=0;
   #20 a=1;
   #20;
  $finish;
  end
  always @(posedge clk) begin
    check_a_and_b:assert (a&&b) $display("a&&b is true");
          else $error("a&&b is false");
   end
endmodule
```

叩特电子Coachip

立即断言 Immediate Assertions

a&&b is ture

asertion_ex.check_a_and_b: started at 15ps failed at 15ps Offending ‘ (a&&b)’

Error: assertion_ex.check_a_and_b: at time 15 ps

a&&b is false

Error: assertion_ex.check_a_and_b: at time 25 ps

a&&b is false

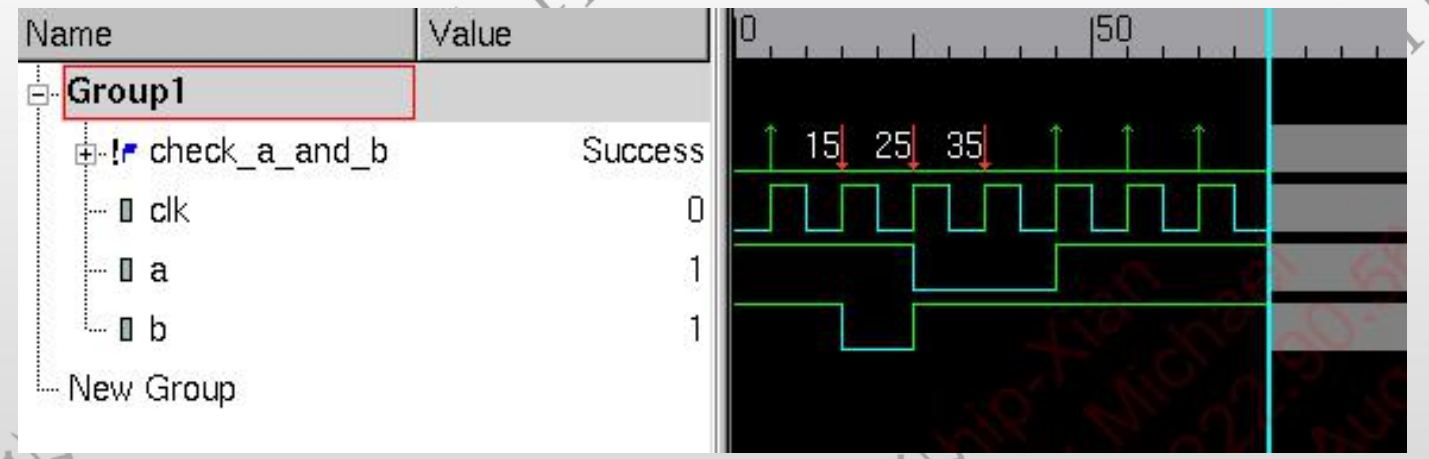
Error: assertion_ex.check_a_and_b: at time 35 ps

a&&b is false

a&&b is ture

a&&b is ture

a&&b is ture



叩特电子Coachip

并发断言 Concurrent Assertions

并发断言检查跨越多个时钟周期的事件序列。

我们可以认为并发断言是一个连续运行的模块，为整个仿真过程检查信号，所以需要在并发断言内指定一个采样时钟。

- 并发断言仅在有时钟周期的情况下出现
- 测试表达式是基于所涉及变量的采样值在时钟边缘进行计算的
- 可以在过程块、module、interface和program块内定义并发断言
- 区别并发断言和立即断言的关键字是**property**

叩特电子Coachip

并发断言 Concurrent Assertions

并发断言的通用格式：

- 断言名：assert property（判断条件）

```
always @(posedge clk) begin
    check_a_and_b:assert (a&&b) $display("a&&b is true");
            else $error("a&&b is false");
end

check_a_and_b:assert property(@(posedge clk) (a&&b)) $display("a&&b is true");
        else $error("a&&b is false");
```

叩特电子Coachip

并发断言 Concurrent Assertions

a&&b is ture

asertion_ex.check_a_and_b: started at 15ps failed at 15ps Offending ‘ (a&&b)’

Error: assertion_ex.check_a_and_b: at time 15 ps

a&&b is false

Error: assertion_ex.check_a_and_b: at time 25 ps

a&&b is false

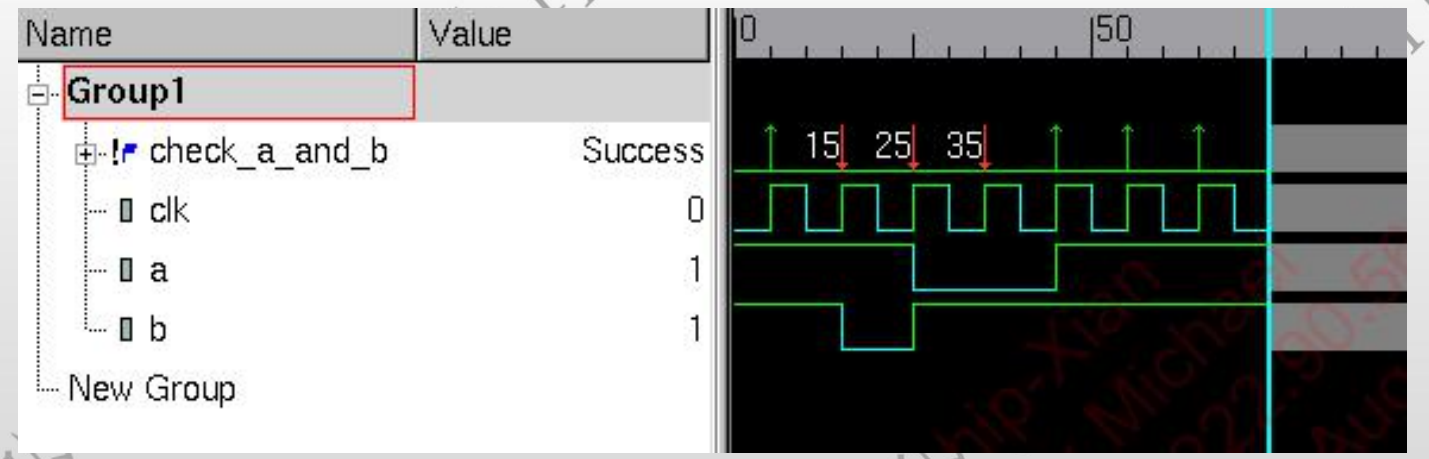
Error: assertion_ex.check_a_and_b: at time 35 ps

a&&b is false

a&&b is ture

a&&b is ture

a&&b is ture



叩特电子Coachip

SV Assertions的层次结构

SVA中可以存在内建的单元，这些单元可以是以下的几种：

- Boolean expressions 布尔表达式

  布尔表达式是组成断言的最小单元，断言可以由多个逻辑事件组成，这些逻辑事件可以是简单的布尔表达式.在SVA中，信号或事件可以使用常用的操作符，如：&&，||，!，^，&等；

- Sequence序列

  sequence是布尔表达式更高一层的单元，一个sequence中可以包含若干个布尔表达式，同时在sequence中可以使用一些新的操作符，如 ## 、重复操作符、序列操作符

- Property 属性

  property是比sequence更高一层的单元，也是构成断言最常用的模块，其中最重要的性质是可以在property中使用蕴含操作符(|-> |=>)；

即特电子Coachip

Sequence序列

在任何设计中，功能总是由多个逻辑事件的组合来表示，这些事件可以是简单的同一个时钟沿被求值得布尔表达式，也可以是经过几个时钟周期的求值事件，SVA用关键字 sequence来表示这些事件，sequence可以让断言易读，复用性高；

sequence具有如下特性：

- 可带参数；
- 可以在 property 中调用；
- 可以使用局部变量；
- 可以定义时钟周期；

sequence的定义格式如下：

```
sequence name_of_sequence(参数);
  ……
endsequence
```

叩特电子Coachip

Sequence序列

```
check_a_and_b:assert property(@(posedge clk) (a&&b)) $display("a&&b is true");
        else $error("a&&b is false");


sequence seq_a_and_b;
  @(posedge clk) a&&b;
endsequence
//在断言的property中调用sequence
check_a_and_b: assert property(seq_a_and_b);
check_a_and_b: assert property(seq_a_and_b) $display("a&&b is true");
        else $error("a&&b is false");
```

叩特电子Coachip

Sequence序列

- 带参数的sequence

```
sequence seq1(signal1,signal2);
    @(posedge clk) signal1&&signal2;
endsequence
//在断言的property中调用sequence
check_a_and_b: assert property(seq1(a,b));


check_a_and_b: assert property(seq1(a,b)) $display("a&&b is true");
            else $error("a&&b is false");
```

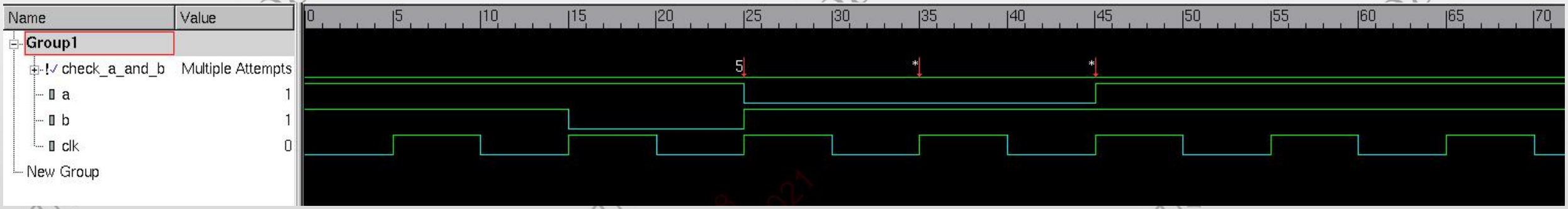
叩特电子Coachip

Sequence序列

带时序关系的sequence ： 在SVA中时钟延时用符号"##"来表示，如"##2"表示延时两个时钟周期；

```
sequence seq2;
  @(posedge clk) a ##2 b ;
endsequence
 //在断言的property中调用sequence
check_a_and_b: assert property(seq2);
```

sequence只有在时钟上升沿到来后检查a是否为1才会继续检查后面的b；



叩特电子Coachip

Sequence序列

sequence内可以调用其它sequence：

```
sequence seq1;
    @(posedge clk) a&&b;
 endsequence
sequence seq2;
    seq1;
endsequence
```

叩特电子Coachip

Property 属性

property是比sequence更高一层的单元，也是构成断言最常用的模块，其中最重要的性质是可以在property中使用蕴含操作符(|-> |=>)；

property的定义格式如下：

```
property name_of_property(参数列表);
  测试表达式或复杂的sequence;
endproperty
```

property就是SVA中需要用来判定的部分，用来模拟过程中被验证的单元，它必须在模拟过程中被断言来发挥作用，SVA提供了关键字 assert 来检查属性，assert的基本语法是：

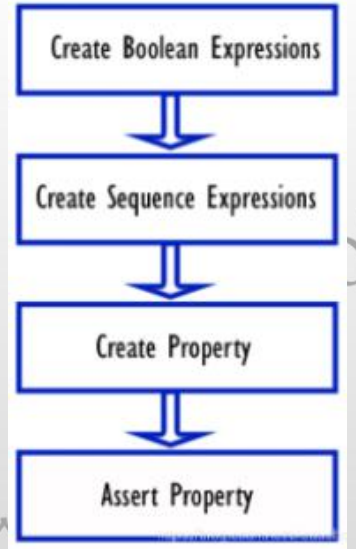
```
assertion_name: assert property(property_name) else $display("SVA error");
```

叩特电子Coachip

property与sequence的异同

- 任何在sequence中的表达式都可以放到property中；
- 任何在property中的表达式也可以搬到sequence中，但是只有在property中才能使用蕴含操作符；
- property中可以例化其他property和sequence，sequence中也可以调用其他的sequence，但是不能例化property；
- property需要用cover /assert/assume 等关键字进行实例化，而sequence直接调用即可；

创建一个断言的步骤：



叩特电子Coachip

SV Assertion 触发判断

- **Clock触发的事件**

  @ (posedge clk) A事件;

- **边沿触发内置函数**

  $rose(a);　//————— 信号上升

  $fell(a);　//————— 信号下降

  $stable(a); //————— 信号值不变

  $changed(a)；//---- return 1 if signal changed

叩特电子Coachip

SV Assertion 触发判断

Assertion Cycle Delay Range(##[])

Operator	Description	Example & Comment
##n	n Cycle Delay	req ##2 grant
##[m:n]	m ~ n Cycle Delay	req ##[1:2] grant
##[*]	##[0:$]	req ##[*] ack //when req, gnt should come eventually
##[+]	##[1:$]	req ##[+] ack //**after req (next cycle)**, ack should come eventually
a[*3]	a ##1 a ##1 a	Consecutive Repetition，a连续3个cycle都成立
a[->2] b	a ##[+] a ##1 b	a成立2次(不要求在连续的2个cycle内)，1个cycle后b成立
a[=2] b	a ##[+] a ##[+] b	a成立2次(不要求在连续的2个cycle内)，b不必匹配连续的时钟周期

Coachip

SV Assertion 触发判断

判断多个事件/信号的行为关系

Operator	Description
intersect(a, b)	断定a和b两个事件同时产生，且同时结束
a within b	断定b事件发生的时间段里包含a事件发生的时间段
a ##2 b a ##[1:3] b a ##[3:$] b	断定a事件发生后2个单位时间内b事件一定会发生 断定a事件发生后1~3个单位时间内b事件一定会发生 断定a事件发生后3个周期时间后b事件一定会发生
c throughout (a ##2 b)	断定在a事件成立到b事件成立的过程中，c事件"一直"成立
@ (posedge clk) a |-> b	断定clk上升沿后，a事件"开始发生"，同时，b事件发生
@ (posedge clk) a |=> b	断定clk上升沿后，a事件开始发生，下一个时钟沿后，b事件开始发生
@ (posedge clk) a |=>##2 b	断定clk上升沿后，a事件开始发生，下三个时钟沿后，b事件开始发生
@ (posedge clk) $past(a,2) == 1'b1	断定a信号在2个时钟周期"以前"，其电平值是1
@ (posedge clk) a [*3] @ (posedge clk) a [*1:3] @ (posedge clk) a [->3]	断定"@ (posedge clk) a"在连续3个时钟周期内都成立 断定"@ (posedge clk) a"在连续1~3个时钟周期内都成立 断定"@ (posedge clk) a"在非连续的3个时钟周期内都成立

Coachip

总线的断言函数

总线就是好多根bit线，共同表示一个数。
SVA提供了多bit状态一起判断的函数，即总线断言函数：

- $onehot(BUS) //————BUS中有且仅有1 bit是高，其他是低。
- $onehot0(BUS) //————BUS中有不超过1 bit是高，也允许全0。
- $isunknown(BUS) //————BUS中存在高阻态或未知态。
- $countones(BUS) == n //————BUS中有且仅有n bits是高，其他是低。
- $past(signal, cycle_num, 1) //----return # of cycle past value of signa

叩特电子Coachip

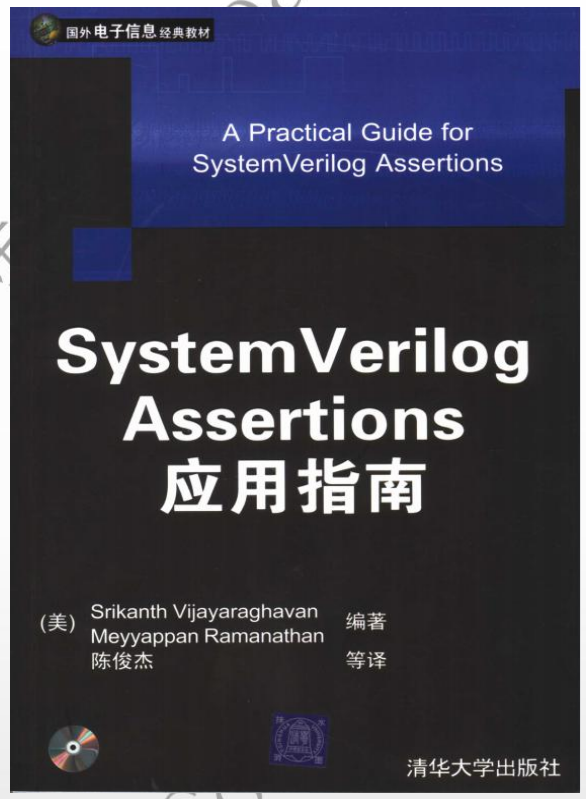
练习

```
module asertion_ex;
  bit clk,a,b;
  always #5 clk = ~clk;   //clock generation
  initial begin
    a=1;     b=1;
    #15 b=0;
    #10 b=1;
        a=0;
    #20 a=1;
    #20;
    $finish;
  end
  always @(posedge clk) check_a_and_b:assert (a&&b);//Immediate assertion
endmodule
```

1，把立即断言改写为并发断言
2，利用sequence改写断言
3，利用带参数的sequence改写断言

叩特电子Coachip

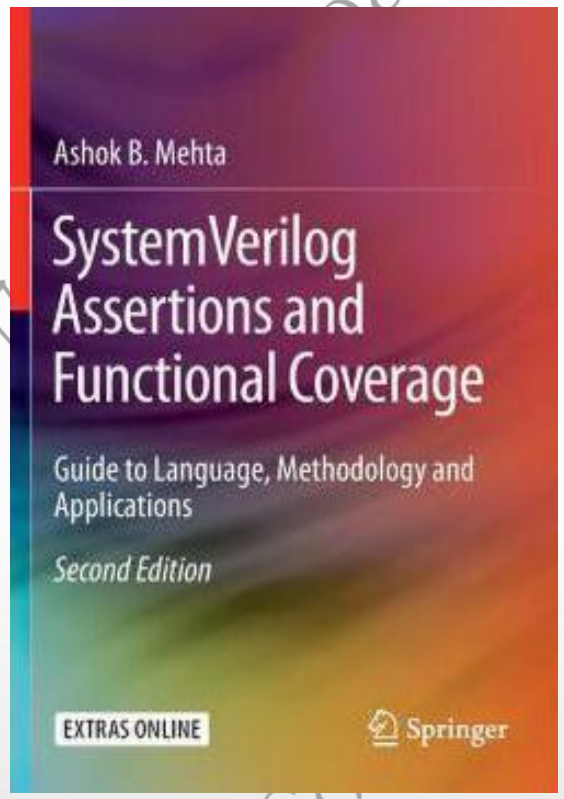
推荐书目



System Verilog Assertions应用指南

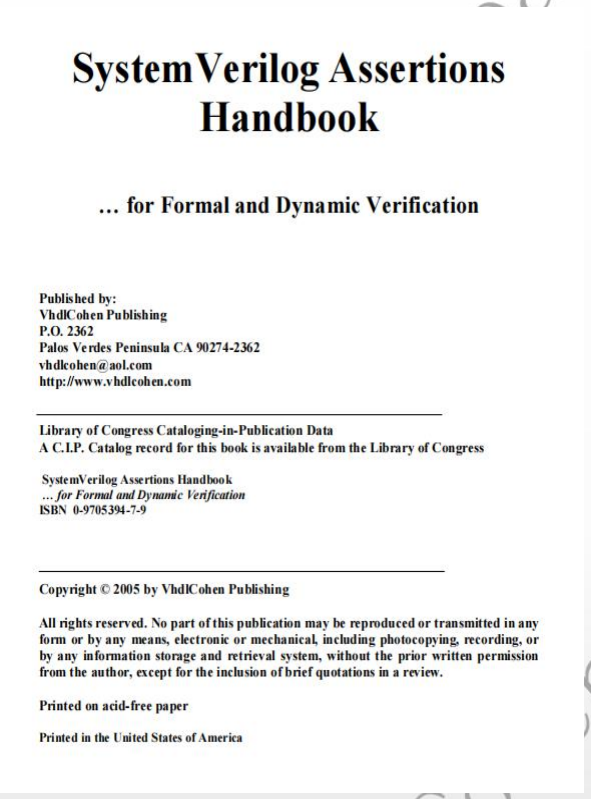
清华大学出版社

陈俊杰 译



System Verilog Assertions and Functional Coverage

Ashok B.Mehta 著



System Verilog Assertions Handbook

即特电子Coachip

Q&A

昂特电子Coachip